Data Structures and Algorithms(23CSE203)

L-T-P: 3-1-2 Credits:5

Lab Manual

Submitted by	
Name	Chethana Nagasai Talluri
Roll No	AV.SC.U4CSE25104
year/Sem/Section	2nd Year/Sem-3/CSE-B
Date of Submission	02/11/2026
Submitted To	
Name	Dr.Raj Kumar
Department	Data structures and algorithms
Designation	Asst.Professor (Sl. Gd.)

Marks	

Index

Week	Title	Page.no	Remark
1	a. A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!". b. Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate p^n (Power), where: • p = Principal growth factor • n = Number of years c. An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. d. In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive Python program to calculate the factorial of a given number. e. Write a recursive Python program to print the first 'n' terms of the Fibonacci series.	5-13	
2	a . Write a python program to implement linear search. b(1). Write a python program to implement binary search with sorted array. b(2).Write a python program to implement binary search without sorted array.	14-17	

Index

Week	Title	Page.no	Remark
3	a.write a python program to implement bubble sort, where the input elements are taken from the user. b.write a python program to implement insertion sort, where the input elements are taken from the user. c.write a python program to implement selection sort, where the input elements are taken from the user	18-23	
4	a) Write a python program to implement merge sort. b) Write a python program to implement quick sort.	24-28	
5	a) Write a python program to implement Singly Linked list with operations 5b) Write a python program to implement Doubly Linked list with operations 5c) Write a python program to implement Circular Linked list with operations	29-51	
6	a) Write a python program to implement stack operations such as push(),pop(),peek().Display() using an array. b) Write a python program to implement stack operations such as push(),pop(),peek().Display() using an Linked list.	52-59	
7	a) Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using array. b) Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using Singly	60-71	

Index

Week	Title	Page.no	Remark
7	Linked list. c) Write a python program to implement a queue and its operations such as enqueue,dequeue,peek and display with a menu driven approach.Implement using Circulated queue.	60-71	
8	a) Write a python program to implement a binary tree with its operations using array. b) Write a python program to implement a binary tree with its operations using linked list.	72-78	
9	a) Write a python program to implement a binary search tree with its operations and traversals.	79-92	
10	(a)Write a python program to implement AVL tree with all its operations.	93-102	
11	(a)Write a python program to implement hashing(separate chaining, Linear probing and Quadratic probing).	103-105	

Week-1

Aim: (a).A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

Code:

```
n=int(input("Enter a number to start the countdown: "))
print("The rocket will launch in.....")

def countdown(n):
    if n == 0:
        print("launch!")
    else:
        print(n)
        countdown(n - 1)

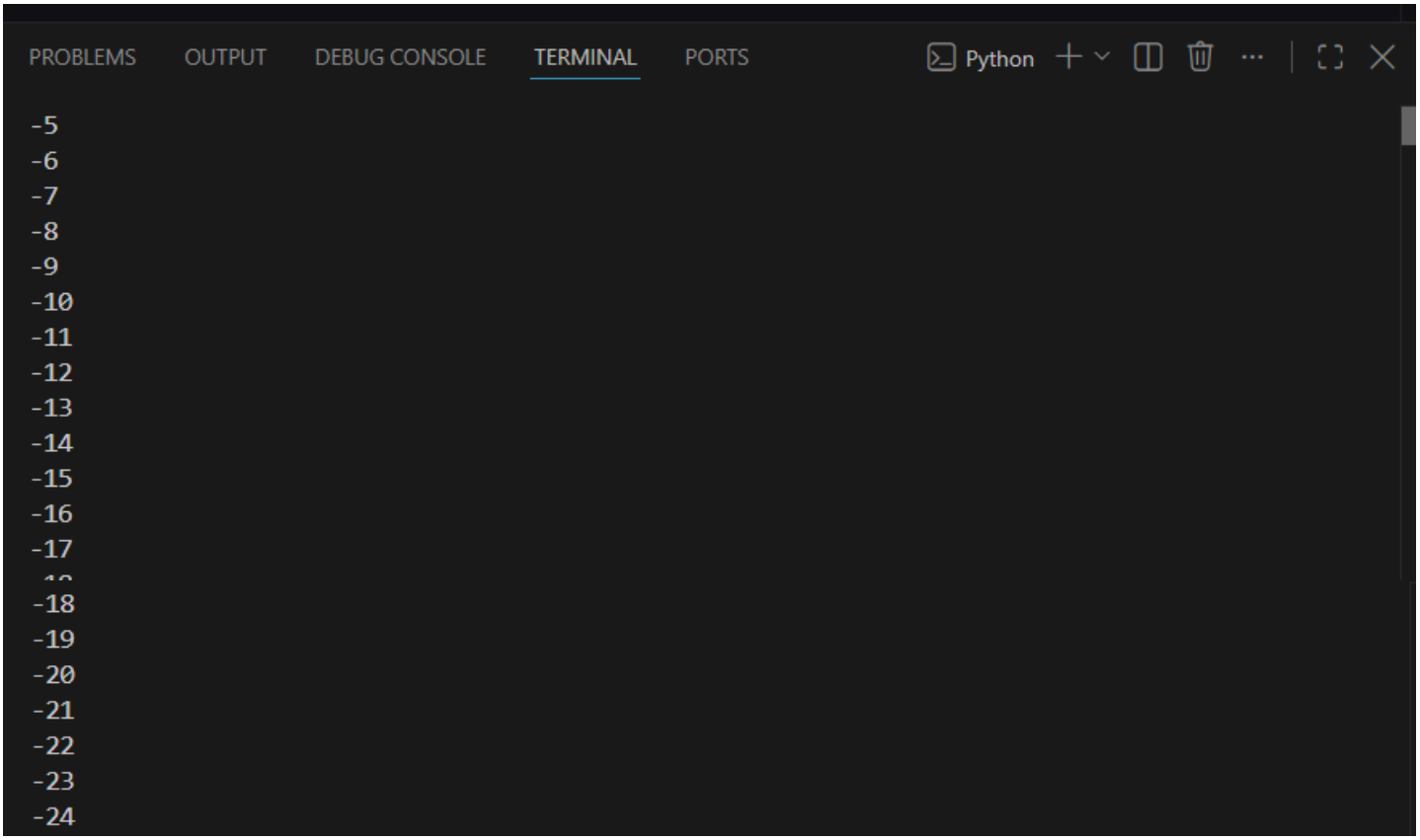
countdown(n)
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppDat
a\Local\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/D
ocuments/SEM-3/DSA/1-1/(a).py"
Enter the countdown number: 5
5
4
3
2
1
Launch!
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

Negative case:



Explanation:

- Prints numbers from n down to 1.
- Uses recursion by calling itself with n-1.
- Base case: n == 1.

Error Table:

s.no	Error	Explanation
1.	Recursive Error	n keeps decreasing: -1, -2, -3..., so it never reaches the base case n=0.

Aim: (b)Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate p^n (Power), where:

- p = Principal growth factor
- n = Number of years

Code:

```
def power(p, n):
  if n == 0:
    return 1
  else:
    return p * power(p, n - 1)

p = float(input("Enter the principal growth factor: "))
n = int(input("Enter the number of years: "))
result = power(p, n)
print("Power =", result)
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-1/(b).py"
Enter the principal growth factor: 1000
Enter the number of years: 2
Power = 1000000.0
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-1/(b).py"
Enter the principal growth factor: 1000
Enter the number of years: -2
Traceback (most recent call last):
  File "c:\Users\cheth\OneDrive\Documents\SEM-3\DSA\l-1\(b).py", line 9, in <module>
    result = power(p, n)
  File "c:\Users\cheth\OneDrive\Documents\SEM-3\DSA\l-1\(b).py", line 5, in power
    return p * power(p, n - 1)
               ~~~~~^^^^^^^^^^
  File "c:\Users\cheth\OneDrive\Documents\SEM-3\DSA\l-1\(b).py", line 5, in power
    return p * power(p, n - 1)
               ~~~~~^^^^^^^^^^
  File "c:\Users\cheth\OneDrive\Documents\SEM-3\DSA\l-1\(b).py", line 5, in power
```

Explanation:

- Defines a recursive function power(p, n).
- Calculates p^n (power of a number).
- Base case: n == 0, returns 1.
- Recursive case: multiplies p by power(p, n-1).

Error table::

s.no	Error	Explanation
1.	recursion error	May cause excessive recursive calls and result in a Recursion Error

Aim: (c)An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Code:

```
e_id=[101,102,103,104,105,106,107]
s_id=int(input("Enter Employee ID from(101-107) to search:"))

def search_employee(e_id,s_id,i):
  if i==len(e_id):
    return -1

  if e_id[i]==s_id:
    return i

  return search_employee(e_id, s_id, i + 1)

result=search_employee(e_id,s_id, 0)
if result != -1:
  print("Employee ID found at position:", result)
else:
  print("Employee not found.")
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-1/(c).py"
Enter Employee ID from(101-107) to search:104
Employee ID found at position: 3
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-1/(c).py"
Enter Employee ID from(101-107) to search:109
Employee not found.
```

Explanation:

- Stores employee IDs in a list.
- Uses recursive linear search to find a given ID.
- Base case: if end of list is reached, returns -1.
- If ID matches, returns its position.
- Otherwise searches the next index recursively.

s.no	Error	Explanation
1.	Value is not found	a is an invalid key type

Aim: (d)In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive Python program to calculate the factorial of a given number.

Code:

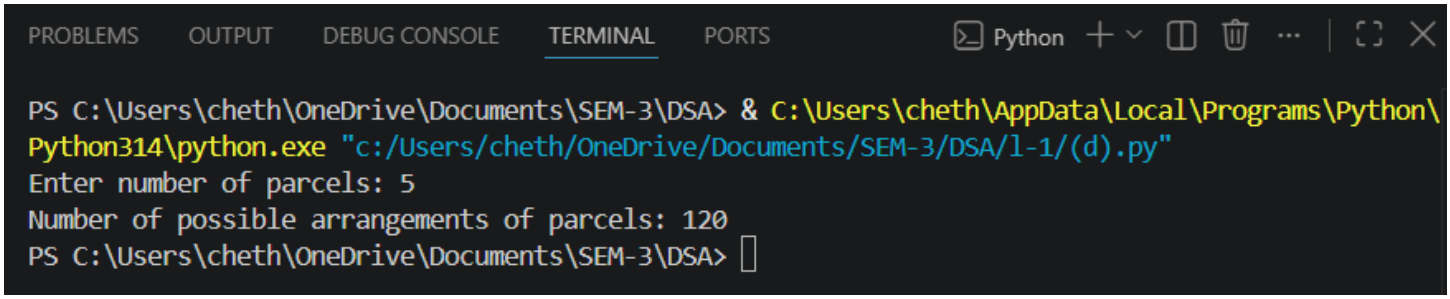
```
def factorial(n):
  if n == 0 or n == 1:
    return 1
  return n * factorial(n - 1)

n = int(input("Enter number of parcels: "))

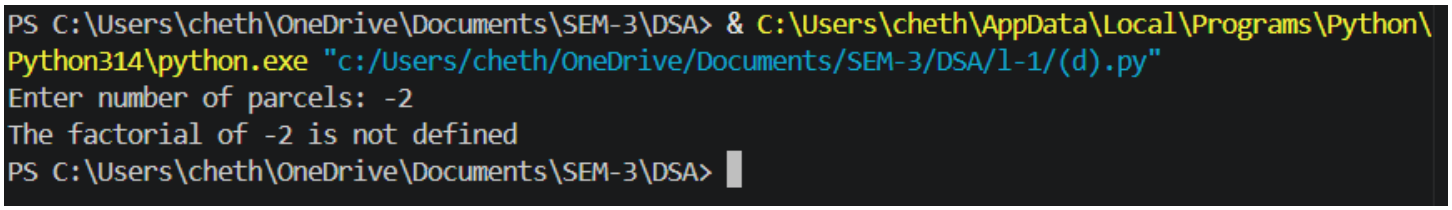
if n < 0:
  print(f"The factorial of {n} is not defined")
else:
  print("Number of possible arrangements of parcels:", factorial(n))
```

Output:

Positive case:



Negative case:



Explanation:

- Defines a recursive function factorial(n).
- Calculates factorial of a number.
- Base case: n == 0 or n == 1, returns 1.
- Recursive case: n × factorial(n-1).

s.no	Error	Correction
1.	Recursion error	May cause excessive recursive calls and result in a Recursion Error.

Aim: (e)Write a recursive Python program to print the first 'n' terms of the Fibonacci series.

Code:

```
def fibonacci(n):
  if n <= 1:
    return n
  return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number: "))

print("Fibonacci Series:")
for i in range(n):
  print(fibonacci(i), end=" ")
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-1/(e).py"
Enter number: 5
Fibonacci Series:
0 1 1 2 3
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-1/(e).py"
Enter number: -3
Fibonacci Series:
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

Explanation:

- Defines a recursive function fibonacci(n).
- Generates Fibonacci numbers.
- Base case: n <= 1, returns n.
- Recursive case: fibonacci(n-1) + fibonacci(n-2).
- Prints Fibonacci series up to n terms.

Week-2

Aim: (a)Write a python program to implement linear search.

Code:

```
arr = [10, 20, 30, 40, 50]

key = int(input("Enter element to search[10-50]: "))

found = 0

for i in range(len(arr)):
  if arr[i] == key:
    print("Element found at position", i + 1)
    found = 1
    break

if found == 0:
  print("Element not found")
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-2/(a).py"
Enter element to search[10-50]: 30
Element found at position 3
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-2/(a).py"
Enter element to search[10-50]: 5
Element not found
```

Explanation:

- Stores elements in an array: [10, 20, 30, 40, 50].
- Takes an element from the user to search.
- Uses a for loop to check each element one by one.
- If the element is found:
  -Prints its position.
  -Sets found = 1.
  -Stops searching using break.
- If the element is not found after checking all elements, prints

Aim: (b).Write a python program to implement binary search without sorted array.

Code:

```
arr = [20, 50, 70, 40, 30, 10, 60]
key = int(input("Enter element to search: "))
arr.sort()
print("Sorted array:", arr)

low = 0
high = len(arr) - 1
found = 0

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = 1
        break
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1

if found == 0:
    print("Element not found")
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-2/(b)2.py"
Enter element to search: 10
Sorted array: [10, 20, 30, 40, 50, 60, 70]
Element found at position 1
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\
Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-2/(b)2.py"
Enter element to search: 80
Sorted array: [10, 20, 30, 40, 50, 60, 70]
Element not found
```

Explanation:

- Stores elements in an unsorted array.
- Accepts an element from the user for searching.
- Sorts the array before performing the search.
- Uses Binary Search to find the element.
- Displays the position if found.
- Displays "Element not found" if the element is absent.

Week-3

Aim: (a)write a python program to implement bubble sort, where the input elements are taken from the user

Code:

```
def bubble_sort(arr):
  n=len(arr)
  for i in range(n-1):
    for j in range(n-i-1):
      if arr[j]>arr[j+1]:
        arr[j],arr[j+1]=arr[j+1],arr[j]
  return arr

n=int(input("Enter the number of elements:"))
arr=[]
print("Enter elements: ")

for i in range(n):
  arr.append(int(input()))
print("Sorted array:", bubble_sort(arr)
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local
\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3
/DSA/l-3/(a).py"
Enter the number of elements:5
Enter elements:
4
9
7
8
2
Sorted array: [2, 4, 7, 8, 9]
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local
\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3
/DSA/l-3/(a).py"
Enter the number of elements:0
Enter elements:
Sorted array: []
```

Explanation:

- Accepts the number of elements from the user.
- Stores the elements in a list.
- Compares adjacent elements and swaps them if they are in the wrong order.
- Repeats the process until the entire list is sorted.
- Displays the sorted array.

Aim: (b)write a python program to implement insertion sort, where the input elements are taken from the user

Code:

```
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
  arr.append(int(input("Enter element: ")))
for i in range(1, n):
  key = arr[i]
  j = i - 1
  while j >= 0 and arr[j] > key:
    arr[j + 1] = arr[j]
    j = j - 1
  arr[j + 1] = key
print("Sorted list:", arr)
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local
\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3
/DSA/1-3/(b).py"
Enter the number of elements: 4
Enter element: 5
Enter element: 7
Enter element: 10
Enter element: 9
Sorted list: [5, 7, 9, 10]
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local
\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3
/DSA/1-3/(b).py"
Enter the number of elements: 0
Sorted list: []
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

Explanation:

- Accepts the number of elements from the user.
- Stores the elements in a list.
- Assumes the first element is already sorted.
- Inserts each remaining element into its correct position in the sorted part of the list.
- Displays the sorted list.

Aim: (c)write a python program to implement selection sort, where the input elements are taken from the user

Code:

```
n = int(input("Enter the number of elements: "))
arr = []

for i in range(n):
  arr.append(int(input("Enter element: ")))

for i in range(n - 1):
  min_index = i

  for j in range(i + 1, n):
    if arr[j] < arr[min_index]:
      min_index = j
  arr[i], arr[min_index] = arr[min_index], arr[i]
print("Sorted list:", arr)
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local
\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3
/DSA/l-3/(c).py"
Enter the number of elements: 3
Enter element: 17
Enter element: 8
Enter element: 25
Sorted list: [8, 17, 25]
```

Negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local
\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3
/DSA/l-3/(c).py"
Enter the number of elements: 0
Sorted list: []
```

Explanation:

- Accepts the number of elements from the user.
- Stores the elements in a list.
- Finds the smallest element from the unsorted portion of the list.
- Swaps the smallest element with the current position.
- Repeats the process until the entire list is sorted.
- Displays the sorted list.

Week-4

Aim: (a)write a python program to implement merge sort, where the input elements are taken from the user

Code:

```
def merge_sort(arr):
  if len(arr) <= 1:
    return arr

  mid = len(arr)//2
  left = arr[:mid]
  right = arr[mid:]

  merge_sort(left)
  merge_sort(right)

  i = j = k = 0
  while i < len(left) and j < len(right):
    if left[i] <= right[j]:
      arr[k] = left[i]
      i += 1
    else:
      arr[k] = right[j]
      j += 1
    k += 1
  while i < len(left):
    arr[k] = left[i]
    i += 1
    k += 1
  while j < len(right):
    arr[k] = right[j]
    j += 1
    k += 1


n = int(input("Enter number of elements:"))
arr = []
```

```
if n > 0:
  print("Enter elements:")
  for i in range(n):
    arr.append(int(input()))
  print("Sorted array:")
  merge_sort(arr)
  print(arr)
else:
  print("No elements to sort.")
```

Output:

positive case:

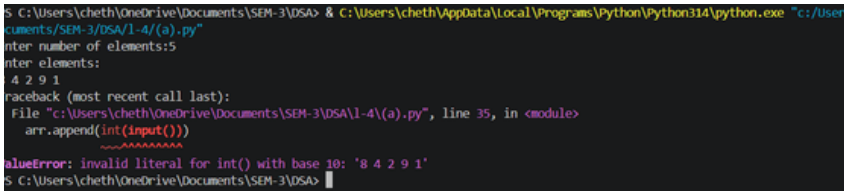
```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-4/(a).py"
Enter number of elements:5
Enter elements:
9
7
5
6
1
Sorted array:
[1, 5, 6, 7, 9]
```

negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-4/(a).py"
Enter number of elements:0
No elements to sort.
```

Explanation:

- This program is used to sort the elements of an array using the Merge Sort technique.
- First, the array is divided into two equal halves.
- Each half is again divided until only one element remains in each sub-array.
- Then the divided sub-arrays are merged back together in sorted order.
- After all merging operations are completed, the entire array becomes sorted.
- Finally, the sorted array is displayed

s.no	Error	correction
1.		arr = list(map(int, input("Enter elements: ").split()))

Aim: (b)write a python program to implement Quick sort, where the input elements are taken from the user

Code:

```
def quick_sort(a, low, high):
  if low < high:
    i = low
    j = high
    pivot = low
    while i < j:
      while i < len(a) and a[i] <= a[pivot]:
        i += 1
      while a[j] > a[pivot]:
        j -= 1
      if i < j:
        a[i], a[j] = a[j], a[i]
    a[j], a[pivot] = a[pivot], a[j]
    quick_sort(a, low, j-1)
    quick_sort(a, j+1, high)


a = list(map(int, input("Enter numbers to sort:").split()))

n = len(a)
if n > 0:
  quick_sort(a, 0, n-1)
  print("Sorted array:")
  print(a)
else:
  print("No elements to sort.")
```

Output:

positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\pyth
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-4/(b).py"
Enter numbers to sort:8 9 3 1 5
Sorted array:
[1, 3, 5, 8, 9]
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\pyth
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-4/(b).py"
Enter numbers to sort:
No elements to sort.
```

Explanation:

- This program is used to sort the elements of an array using the Quick Sort technique.
- The first element of the array is chosen as the pivot.
- The array is rearranged so that elements smaller than the pivot come before it and larger elements come after it.
- The same process is repeated recursively for the left and right parts of the array.
- This continues until all elements are arranged in the correct order.
- Finally, the sorted array is displayed.

Week-5

Aim: (a)write a python program to implement insertion,deletion,travarsing operations using single linked list

Code:

```
class single_linked_list:
  #creating a node
  class Node:
    def __init__(self, data):
      self.data = data
      self.next = None

  #initializing the head
  def __init__(self):
    self.head = None

  #adding a node to the list
  def create(self):
    n = int(input("Enter number of elements:"))
    for i in range(n):
      x = int(input("Enter value:"))
      new = self.Node(x)
      if self.head is None:
        self.head = new
      else:
        temp = self.head
        while temp.next:
          temp = temp.next
        temp.next = new

  #insertion in sll-at the beginning
  def insert_beginning(self, data):
    new = self.Node(data)
    new.next = self.head
    self.head = new
```

```
#insertion in sll-at the end
def insert_end(self, data):
  new = self.Node(data)
  if self.head is None:
    self.head = new
  else:
    temp = self.head
    while temp.next:
      temp = temp.next
    temp.next = new

#insertion in sll-at a given position
def insert_index(self, index, data):
  if index == 0:
    self.insert_beginning(data)
    return
  elif index > self.count():
    print("Invalid Index")
    return
  new = self.Node(data)
  temp = self.head
  for i in range(index - 1):
    temp = temp.next
  new.next = temp.next

#deletion in sll-at the beginning
def delete_beginning(self):
  if self.head is None:
    print("No elements to delete")
  else:
    temp = self.head
    self.head = temp.next
    print("Deleted element = ", temp.data)

#deletion in sll-at the end
def delete_end(self):
  if self.head is None:
```

```
      print("No elements to delete")
    return
  elif self.head.next is None:
    self.head = None
  else:
    temp = self.head
    temp1=temp
    while temp.next:
      temp1 = temp
      temp = temp.next
    temp1.next = None

#deletion in sll-at a given position
def delete_index(self, index):
  if self.head is None:
    print("No elements to delete")
  else:
    temp = self.head
    if temp and temp.data == index:
      self.head = temp.next
      print("Value deleted")
      return
    while temp.next and temp.next.data != index:
      temp = temp.next
    if temp.next is None:
      print("Value not present")
    else:
      temp.next = temp.next.next
      print("Value deleted")

#displaying the elements of sll
def display(self):
  if self.head is None:
    print("No elements to display")
  else:
    temp = self.head
    while temp:
```

```
        print(temp.data, end=" -> ")
        temp = temp.next
      print("None")

  #counting the number of elements in sll
  def count(self):
    if self.head is None:
      print("No elements to count")
    else:
      c = 0
      temp = self.head
      while temp:
        c += 1
        temp = temp.next
      return c

sll = single_linked_list()
while True:
  print("1. Create a single linked list")
  print("2. Insert at the beginning")
  print("3. Insert at the end")
  print("4. Insert at a given position")
  print("5. Delete at the beginning")
  print("6. Delete at the end")
  print("7. Delete at a given position")
  print("8. Display the list")
  print("9. Count the elements")
  print("10. Exit")

  choice = int(input("Enter your choice:"))
  if choice == 1:
    sll.create()
  elif choice == 2:
    data = int(input("Enter value to insert:"))
    sll.insert_beginning(data)
  elif choice == 3:
    X = int(input("Enter value to insert:"))
```

```
    sll.insert_end(X)
  elif choice == 4:
    X = int(input("Enter value to insert:"))
    index = int(input("Enter index to insert:"))
    sll.insert_index(index, X)
  elif choice == 5:
    sll.delete_beginning()
  elif choice == 6:
    sll.delete_end()
  elif choice == 7:
    index = int(input("Enter value to delete:"))
    sll.delete_index(index)
  elif choice == 8:
    sll.display()
  elif choice == 9:
    count = sll.count()
    print("Number of nodes = ", count)
  elif choice == 10:
    break
  else:
    print("Invalid choice")
```

Output:

positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\pyth
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-5/(a).py"
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:1
Enter number of elements:5
Enter value:101
Enter value:102
Enter value:103
Enter value:104
Enter value:105
```

```
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:2
Enter value to insert:100
```

```
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:3
Enter value to insert:106
```

```
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:4
Enter value to insert:107
Enter index to insert:4
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:5
Deleted element =  100
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:6
```

```
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:7
Enter value to delete:107
Value not present
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:8
101 -> 102 -> 103 -> 104 -> 105 -> None
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:9
Number of nodes =  5
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:10
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

negative case:

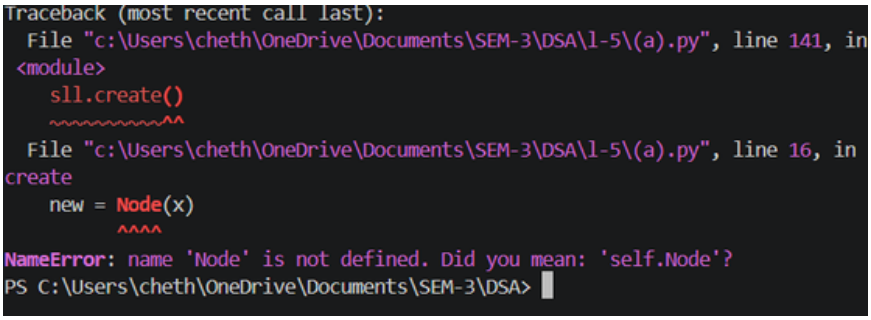
```
1. Create a single linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete at the beginning
6. Delete at the end
7. Delete at a given position
8. Display the list
9. Count the elements
10. Exit
Enter your choice:11
Invalid choice
```

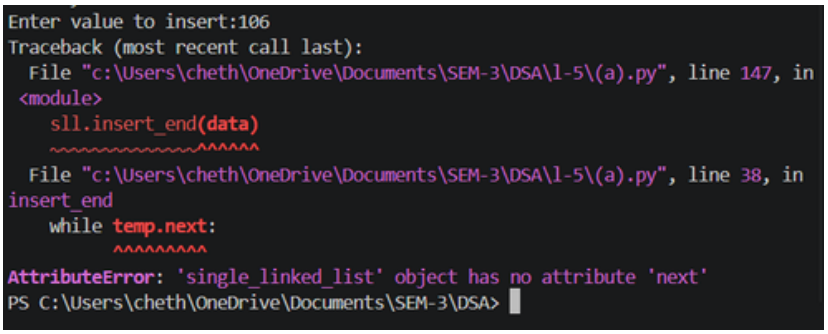
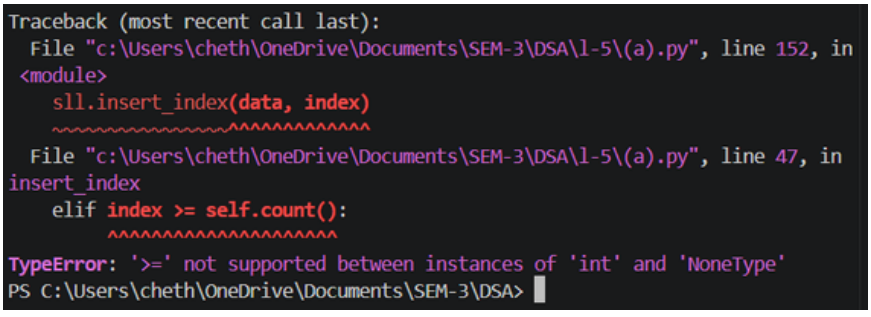
Explanation:

- This program is used to create and perform operations on a Single Linked List.
- Each node contains data and a link to the next node.
- It allows insertion of nodes at the beginning, end, and a specified position.
- It allows deletion of nodes from the beginning, end, and by a given value.
- It can display all elements present in the list.
- It can count and display the total number of nodes in the list

Error table:

s.no	error	correction

1.		class Node:
		def __init__(self, data):
		self.data = data
		self.next = None

2.		Initialize temp = self.head.
3.		Add return count at the end of count().

Aim: (b)write a python program to implement insertion,deletion,travarsing operations using double linked list

Code:

```
class double_linked_list:
  #creating a node
  class Node:
    def __init__(self, data):
      self.data = data
      self.prev = None
      self.next = None

  #initializing the head
  def __init__(self):
    self.head = None

  #adding a node to the list
  def create(self):
    n = int(input("Enter number of elements:"))
    for i in range(n):
      x = int(input("Enter value:"))
      new = self.Node(x)
      if self.head is None:
        self.head = new
      else:
        temp = self.head
        while temp.next:
          temp = temp.next
        temp.next = new
        new.prev = temp

  #insertion in dll-at the beginning
  def insert_beginning(self, data):
    new = self.Node(data)
    if self.head is None:
      self.head = new
    else:
```

```
    new.next = self.head
    self.head.prev = new
    self.head = new
  print("Node inserted at the beginning.")

#insertion in dll-at the end
def insert_end(self, data):
  new = self.Node(data)
  if self.head is None:
    self.head = new
  else:
    temp = self.head
    while temp.next:
      temp = temp.next
    temp.next = new
    new.prev = temp
  print("Node inserted at the end.")

#insertion in dll-at a given position
def insert_index(self, index, data):
  if index < 0:
    print("Invalid Index")
  elif index == 0:
    self.insert_beginning(data)
  else:
    new = self.Node(data)
    temp = self.head
    for i in range(index - 1):
      if temp is None:
        print("Invalid Index")
        return
      temp = temp.next
    new.next = temp.next
    new.prev = temp
    if temp.next:
      temp.next.prev = new
    temp.next = new
```

```
  print("Node inserted at index", index)

#deletion in dll-at the beginning
def delete_beginning(self):
  if self.head is None:
    print("No elements to delete")
  elif self.head.next is None:
    self.head = None
    print("Deleted the only element in the list.")
  else:
    self.head.next.prev = None
    self.head = self.head.next
    print("Deleted element at the beginning.")

#deletion in dll-at the end
def delete_end(self):
  if self.head is None:
    print("No elements to delete")
  elif self.head.next is None:
    self.head = None
    print("Deleted the only element in the list.")
  else:
    temp = self.head
    while temp.next:
      temp = temp.next
    temp.prev.next = None
    temp.next = None
    print("Deleted element at the end.")

#deletion in dll-at a given position
def delete_index(self, index):
  if self.head is None:
    print("No elements to delete")
  elif index < 0:
    print("Invalid Index")
  elif index == 0:
    self.delete_beginning()
```

```
    else:
      temp = self.head
      for i in range(index):
        if temp is None:
          print("Invalid Index")
          return
        temp = temp.next
      if temp is None:
        print("Invalid Index")
      elif temp.next:
        temp.next.prev = temp.prev
        temp.prev.next = temp.next
      else:
        temp.prev.next = None
        temp.next = None
      print("Deleted element at index", index)

#displaying the elements of dll in forward direction
def display_forward(self):
  temp = self.head
  while temp:
    print(temp.data, end=" <-> ")
    temp = temp.next
  print("None")

#displaying the elements of dll in backward direction
def display_backward(self):
  temp = self.head
  if temp is None:
    print("None")
    return
  while temp.next:
    temp = temp.next
  while temp:
    print(temp.data, end=" <-> ")
    temp = temp.prev
  print("None")
```

```
dll = double_linked_list()
while True:
  print("1. Create a new list")
  print("2. Insert at the beginning")
  print("3. Insert at the end")
  print("4. Insert at a given index")
  print("5. Delete from the beginning")
  print("6. Delete from the end")
  print("7. Delete from a given index")
  print("8. Display forward")
  print("9. Display backward")
  print("10. Exit")

  choice = int(input("Enter your choice: "))

  if choice == 1:
    dll.create()
  elif choice == 2:
    data = int(input("Enter value to insert:"))
    dll.insert_beginning(data)
  elif choice == 3:
    data = int(input("Enter value to insert:"))
    dll.insert_end(data)
  elif choice == 4:
    index = int(input("Enter index to insert:"))
    data = int(input("Enter value to insert:"))
    dll.insert_index(index, data)
  elif choice == 5:
    dll.delete_beginning()
  elif choice == 6:
    dll.delete_end()
  elif choice == 7:
    index = int(input("Enter index to delete:"))
    dll.delete_index(index)
  elif choice == 8:
    dll.display_forward()
  elif choice == 9:
```

```
    dll.display_backward()
  elif choice == 10:
    break
  else:
    print("Invalid choice")
```

Output:

positive case:

```
                                                > & C:\Users\cheth\AppData\Local\Programs\Python\Python314\pyth
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-5/(b).py"
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 1
Enter number of elements:3
Enter value:101
Enter value:102
Enter value:103
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 2
Enter value to insert:100
Node inserted at the beginning.
```

```
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 3
Enter value to insert:104
Node inserted at the end.
```

```
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 4
Enter index to insert:2
Enter value to insert:105
Node inserted at index 2
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 5
Deleted element at the beginning.
```

```
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 6
Deleted element at the end.
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 7
Enter index to delete:1
Deleted element at index 1
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 8
101 <-> 102 <-> 103 <-> None
```

```
10. Exit
Enter your choice: 8
101 <-> 102 <-> 103 <-> None
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 9
103 <-> 102 <-> 101 <-> None
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 10
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

negative case:

```
1. Create a new list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given index
5. Delete from the beginning
6. Delete from the end
7. Delete from a given index
8. Display forward
9. Display backward
10. Exit
Enter your choice: 11
Invalid choice
```

Explanation:

- This program is used to create and perform operations on a Double Linked List.
- Each node contains data, a link to the previous node, and a link to the next node.
- It allows insertion of nodes at the beginning, end, and a specified index.
- It allows deletion of nodes from the beginning, end, and a specified index.
- It can display the list in both forward and backward directions.
- It provides a menu-driven interface for performing all operations.

Aim: (c)write a python program to implement insertion,deletion,travarsing operations using circular linked list

Code:

```
class circular_linked_list:
  #creating a node
  class Node:
    def __init__(self, data):
      self.data = data
      self.next = None

  #initializing the head and tail
  def __init__(self):
    self.head = None
    self.tail = None

  #adding a node to the list
  def create(self):
    n = int(input("Enter number of elements:"))
    for i in range(n):
      x = int(input("Enter value:"))
      new = self.Node(x)
      if self.head is None:
        self.head = new
        self.tail = new
        new.next = self.head
      else:
        self.tail.next = new
        self.tail = new
        self.tail.next = self.head

  #insertion in cll-at the beginning
  def insert_beginning(self, data):
    new = self.Node(data)
    if self.head is None:
      self.head = new
      self.tail = new
```

```
    new.next = self.head
  else:
    new.next = self.head
    self.head = new
    self.tail.next = self.head

#insertion in cll-at the end
def insert_end(self, data):
  new = self.Node(data)
  if self.head is None:
    self.head = new
    self.tail = new
    new.next = self.head
  else:
    self.tail.next = new
    self.tail = new
    self.tail.next = self.head

#insertion in cll-at a given position
def insert_position(self, position, data):
  if position == 0:
    self.insert_beginning(data)
    return

  if self.head is None or position < 0:
    print("Invalid position")
    return

  new = self.Node(data)
  temp = self.head
  for i in range(position - 1):
    temp = temp.next
    if temp == self.head:
      print("Invalid position")
      return

  new.next = temp.next
```

```
  temp.next = new

  if temp == self.tail:
    self.tail = new

#deletion in cll-at the beginning
def delete_beginning(self):
  if self.head is None:
    print("No elements to delete")
    return
  elif self.head == self.tail:
    self.head = None
    self.tail = None
  else:
    temp = self.head
    self.head = temp.next
    self.tail.next = self.head

#deletion in cll-at the end
def delete_end(self):
  if self.head is None:
    print("No elements to delete")
    return
  elif self.head == self.tail:
    self.head = None
    self.tail = None
    return

  temp = self.head
  while temp.next != self.tail:
    temp = temp.next
  temp.next = self.head
  self.tail = temp

#traversing the list
def traverse(self):
  if self.head is None:
```

```
      print("List is empty")
      return
    temp = self.head
    while True:
      print(temp.data, end="->")
      temp = temp.next
      if temp == self.head:
        break
    print("(back to head)")

  #display head and tail
  def display_head_tail(self):
    if self.head is None:
      print("List is empty")
    else:
      print("Head = ", self.head.data)
      print("Tail = ", self.tail.data)

cll = circular_linked_list()
while True:
  print("1. Create a circular linked list")
  print("2. Insert at the beginning")
  print("3. Insert at the end")
  print("4. Insert at a given position")
  print("5. Delete from the beginning")
  print("6. Delete from the end")
  print("7. Traverse the list")
  print("8. Display head and tail")
  print("9. Exit")
  choice = int(input("Enter your choice:"))

  if choice == 1:
    cll.create()
  elif choice == 2:
    data = int(input("Enter value to insert:"))
    cll.insert_beginning(data)
  elif choice == 3:
```

```
    data = int(input("Enter value to insert:"))
    cll.insert_end(data)
  elif choice == 4:
    position = int(input("Enter position to insert:"))
    data = int(input("Enter value to insert:"))
    cll.insert_position(position, data)
  elif choice == 5:
    cll.delete_beginning()
  elif choice == 6:
    cll.delete_end()
  elif choice == 7:
    cll.traverse()
  elif choice == 8:
    cll.display_head_tail()
  elif choice == 9:
    break
  else:
    print("Invalid choice")
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-5/(c).py"
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:1
Enter number of elements:3
Enter value:101
Enter value:102
Enter value:103
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:2
Enter value to insert:100
```

```
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:3
Enter value to insert:104
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:4
Enter position to insert:4
Enter value to insert:105
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:5
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:6
```

```
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:7
101->102->103->105->(back to head)
```

```
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:8
Head =  101
Tail =  105
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:9
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

negative case:

```
1. Create a circular linked list
2. Insert at the beginning
3. Insert at the end
4. Insert at a given position
5. Delete from the beginning
6. Delete from the end
7. Traverse the list
8. Display head and tail
9. Exit
Enter your choice:10
Invalid choice
```

Explanation:

- This program is used to create and perform operations on a Circular Linked List.
- In a circular linked list, the last node points back to the first node.
- It allows insertion of nodes at the beginning, end, and a specified position.
- It allows deletion of nodes from the beginning and end of the list.
- It can traverse and display all nodes in the circular sequence.
- It can display the head and tail nodes of the list.
- It provides a menu-driven interface for performing various circular linked list operations.

Week-6

Aim: (a)Write a python program to implement stack operations such a push,pop,peek and display using Arrays

Code:

```
class stack:
  #initializing stack
  def __init__(self, size):
    self.size = size
    self.stack = [None] * size
    self.top = -1

  #push operation
  def push(self, data):
    if self.top == self.size - 1:
      print("Stack Overflow")
    else:
      self.top += 1
      self.stack[self.top] = data
      print(f"{data} pushed to stack.")

  #pop operation
  def pop(self):
    if self.top == -1:
      print("Stack Underflow")
    else:
      item = self.stack[self.top]
      self.stack[self.top] = None
      self.top -= 1
      print(f"{item} popped from stack.")

  #peek operation
  def peek(self):
    if self.top == -1:
      print("Stack is empty")
    else:
      print(f"Top element is {self.stack[self.top]}.")
```

```
    #display operation
  def display(self):
    if self.top == -1:
      print("Stack is empty")
    else:
      print("Stack elements are:")
      for i in range(self.top, -1, -1):
        print(self.stack[i])

#creating stack object
size = int(input("Enter size of stack:"))
s = stack(size)
while True:
  print("1. push")
  print("2. pop")
  print("3. peek")
  print("4. display")
  print("5. exit")

  choice = int(input("Enter your choice: "))

  if choice == 1:
    data = int(input("Enter value to push:"))
    s.push(data)
  elif choice == 2:
    s.pop()
  elif choice == 3:
    s.peek()
  elif choice == 4:
    s.display()
  elif choice == 5:
    break
  else:
    print("Invalid choice.")
```

Output:

positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\pyth
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-6/(a).py"
Enter size of stack:5
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 1
Enter value to push:10
10 pushed to stack.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 1
Enter value to push:20
20 pushed to stack.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 1
Enter value to push:30
30 pushed to stack.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 2
30 popped from stack.
```

```
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 3
Top element is 20.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 4
Stack elements are:
20
10
```

```
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 5
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

negative case:

```
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 6
Invalid choice.
```

Explanation:

- This program is used to implement a Stack using an array.
- The stack is created with a fixed size specified by the user.
- It supports the push operation to insert an element at the top of the stack.
- It supports the pop operation to remove the top element from the stack.
- It supports the peek operation to view the top element without removing it.
- It displays all elements of the stack from top to bottom.
- It handles Stack Overflow and Stack Underflow conditions.
- It provides a menu-driven interface to perform stack operations.

Aim: (b)Write a python program to implement stack operations such a push,pop,peek and display using Linked lists

Code:

```
class stack:
  #creating node
  class Node:
    def __init__(self, data):
      self.data = data
      self.next = None

  def __init__(self):
    self.top = None

  #push operation
  def push(self, data):
    new = self.Node(data)
    new.next = self.top
    self.top = new
    print(f"{data} pushed to stack.")

  #pop operation
  def pop(self):
    if self.top is None:
      print("Stack Underflow")
      return None
    else:
      temp = self.top
      self.top = self.top.next
      print(f"{temp.data} popped from stack.")
      return temp.data

  #peek operation
  def peek(self):
    if self.top is None:
      print("Stack is empty")
    else:
```

```
    print(f"Top element is {self.top.data}.")

  #display operation
  def display(self):
    if self.top is None:
      print("Stack is empty")
    else:
      temp = self.top
      print("Stack elements are:")
      while temp is not None:
        print(temp.data)
        temp = temp.next

s = stack()
while True:
  print("1. push")
  print("2. pop")
  print("3. peek")
  print("4. display")
  print("5. exit")

  choice = int(input("Enter your choice: "))

  if choice == 1:
    data = int(input("Enter value to push:"))
    s.push(data)
  elif choice == 2:
    s.pop()
  elif choice == 3:
    s.peek()
  elif choice == 4:
    s.display()
  elif choice == 5:
    break
  else:
    print("Invalid choice. Please try again.")
```

Output:

Positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-6/(b).py"
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 1
Enter value to push:10
10 pushed to stack.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 1
Enter value to push:20
20 pushed to stack.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 1
Enter value to push:30
30 pushed to stack.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 2
30 popped from stack.
```

```
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 3
Top element is 20.
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 4
Stack elements are:
20
10
```

```
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 5
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

Positive case:

```
1. push
2. pop
3. peek
4. display
5. exit
Enter your choice: 6
Invalid choice. Please try again.
```

Explanation:

- This program is used to implement a Stack using a Linked List.
- Each node contains data and a link to the next node.
- It supports the push operation to insert a new node at the top of the stack.
- It supports the pop operation to remove the top node from the stack.
- It supports the peek operation to view the top element of the stack.
- It displays all elements present in the stack.
- It handles the Stack Underflow condition when the stack is empty.
- It provides a menu-driven interface to perform stack operations.

Week-7

Aim: (a)Write a python program to implement queue operations such a enqueue, deque,peek and display with menu driven approach using Regular queue and arrays

Code:

```
class queuearray:

  def __init__(self, size):
    self.size = size
    self.queue = [None] * size
    self.front = -1
    self.rear = -1

  #enqueue operation
  def enqueue(self, data):
    if self.rear == self.size - 1:
      print("Queue Overflow")
    else:
      if self.front == -1:
        self.front = 0
      self.rear += 1
      self.queue[self.rear] = data
      print(f"{data} enqueued to queue.")

  #dequeue operation
  def dequeue(self):
    if self.front == -1 or self.front > self.rear:
      print("Queue Underflow")
    else:
      item = self.queue[self.front]
      self.queue[self.front] = None
      self.front += 1
      print(f"{item} dequeued from queue.")

      if self.front > self.rear:
        self.front = -1
        self.rear = -1
```

```
  #peek operation
  def peek(self):
    if self.front == -1:
      print("Queue is empty")
    else:
      print(f"Front element is {self.queue[self.front]}.")

  #display operation
  def display(self):
    if self.front == -1:
      print("Queue is empty")
    else:
      print("Queue elements are:")
      for i in range(self.front, self.rear + 1):
        print(self.queue[i])

class queuesll:

  class Node:
    def __init__(self, data):
      self.data = data
      self.next = None

  def __init__(self):
    self.front = None
    self.rear = None

  #enqueue operation
  def enqueue(self, data):
    new = self.Node(data)
    if self.rear is None:
      self.front = new
      self.rear = new
    else:
      self.rear.next = new
      self.rear = new
    print(f"{data} enqueued to queue.")
```

```
  #dequeue operation
  def dequeue(self):
    if self.front is None:
      print("Queue Underflow")
    else:
      item = self.front.data
      self.front = self.front.next
      if self.front is None:
        self.rear = None
      print(f"{item} dequeued from queue.")

  #peek operation
  def peek(self):
    if self.front is None:
      print("Queue is empty")
    else:
      print(f"Front element is {self.front.data}.")

  #display operation
  def display(self):
    if self.front is None:
      print("Queue is empty")
    else:
      temp = self.front
      print("Queue elements are:")
      while temp is not None:
        print(temp.data)
        temp = temp.next

q_arr = queuearray(int(input("Enter size of queue(array):")))
q_sll = queuesll()
while True:
  print("1. Enqueue in array-based queue")
  print("2. Dequeue from array-based queue")
  print("3. Peek in array-based queue")
  print("4. Display array-based queue")
  print("5. Enqueue in linked list-based queue")
```

```
print("6. Dequeue from linked list-based queue")
print("7. Peek in linked list-based queue")
print("8. Display linked list-based queue")
print("9. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
  data = int(input("Enter value to enqueue:"))
  q_arr.enqueue(data)
elif choice == 2:
  q_arr.dequeue()
elif choice == 3:
  q_arr.peek()
elif choice == 4:
  q_arr.display()
elif choice == 5:
  data = int(input("Enter value to enqueue:"))
  q_sll.enqueue(data)
elif choice == 6:
  q_sll.dequeue()
elif choice == 7:
  q_sll.peek()
elif choice == 8:
  q_sll.display()
elif choice == 9:
  break
else:
  print("Invalid choice")
```

Output:

positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs\Python\Python314\pyth
on.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-7/(a).py"
Enter size of queue(array):5
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 1
Enter value to enqueue:10
10 enqueued to queue.
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 1
Enter value to enqueue:20
20 enqueued to queue.
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 1
Enter value to enqueue:30
30 enqueued to queue.
```

```
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 2
10 dequeued from queue.
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 3
Front element is 20.
```

```
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 4
Queue elements are:
20
30
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 5
Enter value to enqueue:10
10 enqueued to queue.
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 5
Enter value to enqueue:20
20 enqueued to queue.
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 5
Enter value to enqueue:30
30 enqueued to queue.
```

```
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 6
10 dequeued from queue.
```

```
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 7
Front element is 20.
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 8
Queue elements are:
20
30
```

```
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 9
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

Negative case:

```
1. Enqueue in array-based queue
2. Dequeue from array-based queue
3. Peek in array-based queue
4. Display array-based queue
5. Enqueue in linked list-based queue
6. Dequeue from linked list-based queue
7. Peek in linked list-based queue
8. Display linked list-based queue
9. Exit
Enter your choice: 10
Invalid choice
```

Explanation:

using arrays:

- This program is used to implement a Queue using an array.
- The queue is created with a fixed size entered by the user.
- It supports the enqueue operation to insert an element at the rear of the queue.
- It supports the dequeue operation to remove an element from the front of the queue.

- It supports the peek operation to view the front element without removing it.
- It displays all elements present in the queue.
- It handles Queue Overflow and Queue Underflow conditions.
- It follows the FIFO (First In First Out) principle.

using single linked list:

- This program is used to implement a Queue using a Linked List.
- Each node contains data and a link to the next node.
- It supports the enqueue operation to insert a new node at the rear of the queue.
- It supports the dequeue operation to remove a node from the front of the queue.
- It supports the peek operation to view the front element.
- It displays all elements present in the queue.
- It handles the Queue Underflow condition when the queue is empty.
- It follows the FIFO (First In First Out) principle.

Aim: (b)Write a python program to implement queue operations such a enqueue, deque,peek and display with menu driven approach using Circular queue

Code:

```
class circularqueue:

  def __init__(self, size):
    self.size = size
    self.queue = [None] * size
    self.front = -1
    self.rear = -1

  #enqueue operation
  def enqueue(self, data):
    if (self.rear + 1) % self.size == self.front:
      print("Queue Overflow")
    elif self.front == -1:
      self.front = 0
      self.rear = 0
      self.queue[self.rear] = data
      print(f"{data} enqueued to queue.")
    else:
      self.rear = (self.rear + 1) % self.size
      self.queue[self.rear] = data
      print(f"{data} enqueued to queue.")

  #dequeue operation
  def dequeue(self):
    if self.front == -1:
      print("Queue Underflow")
    elif self.front == self.rear:
      item = self.queue[self.front]
      self.queue[self.front] = None
      self.front = -1
      self.rear = -1
      print(f"{item} dequeued from queue.")
    else:
```

```
      item = self.queue[self.front]
      self.queue[self.front] = None
      self.front = (self.front + 1) % self.size
      print(f"{item} dequeued from queue.")

  #peek operation
  def peek(self):
    if self.front == -1:
      print("Queue is empty")
    else:
      print(f"Front element is {self.queue[self.front]}.")

  #display operation
  def display(self):
    if self.front == -1:
      print("Queue is empty")
    else:
      print("Queue elements are:")
      i = self.front
      while True:
        print(self.queue[i])
        if i == self.rear:
          break
        i = (i + 1) % self.size

size = int(input("Enter size of circular queue:"))
cq = circularqueue(size)
while True:
  print("1. enqueue")
  print("2. dequeue")
  print("3. peek")
  print("4. display")
  print("5. exit")

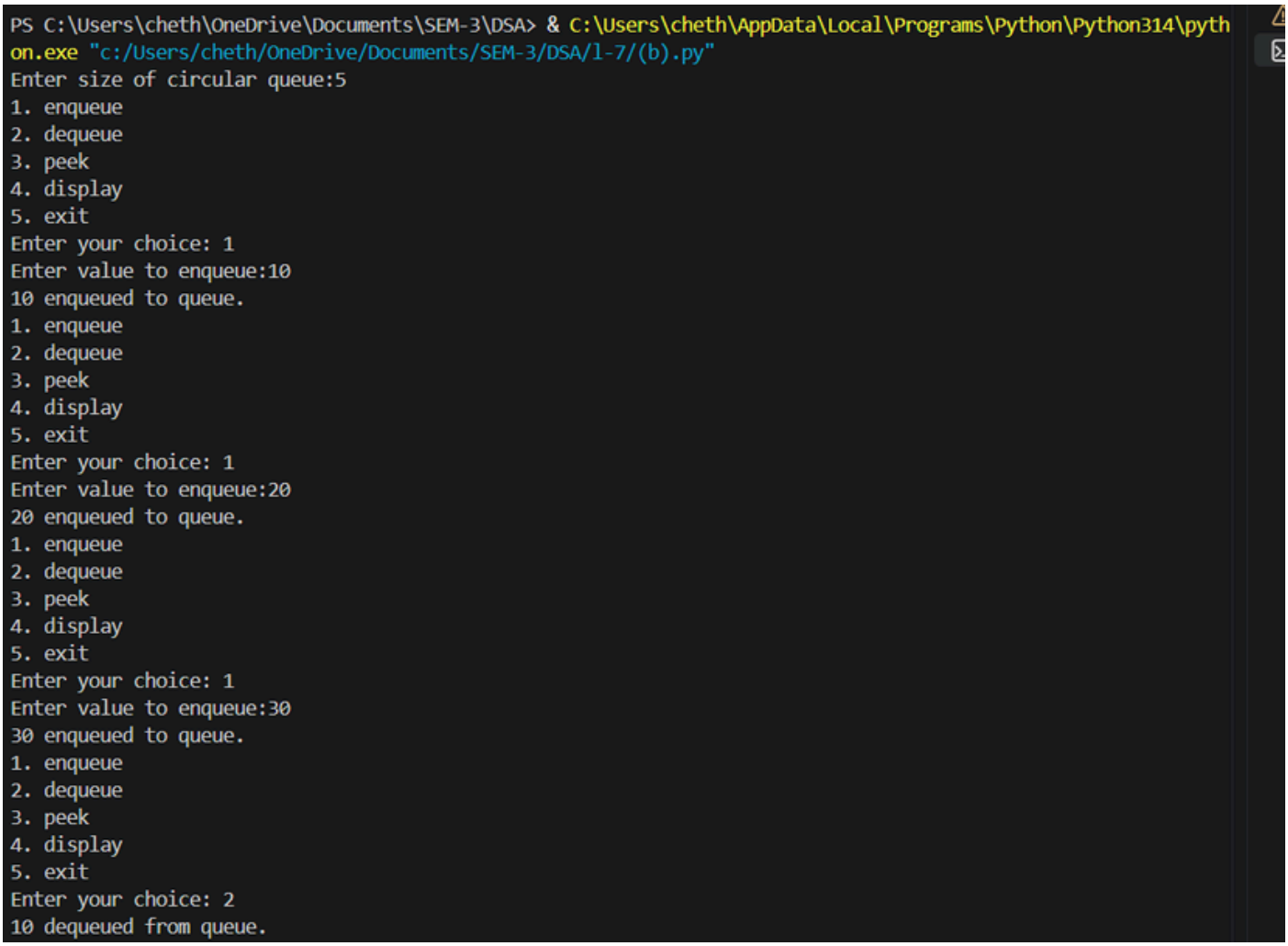
  choice = int(input("Enter your choice: "))

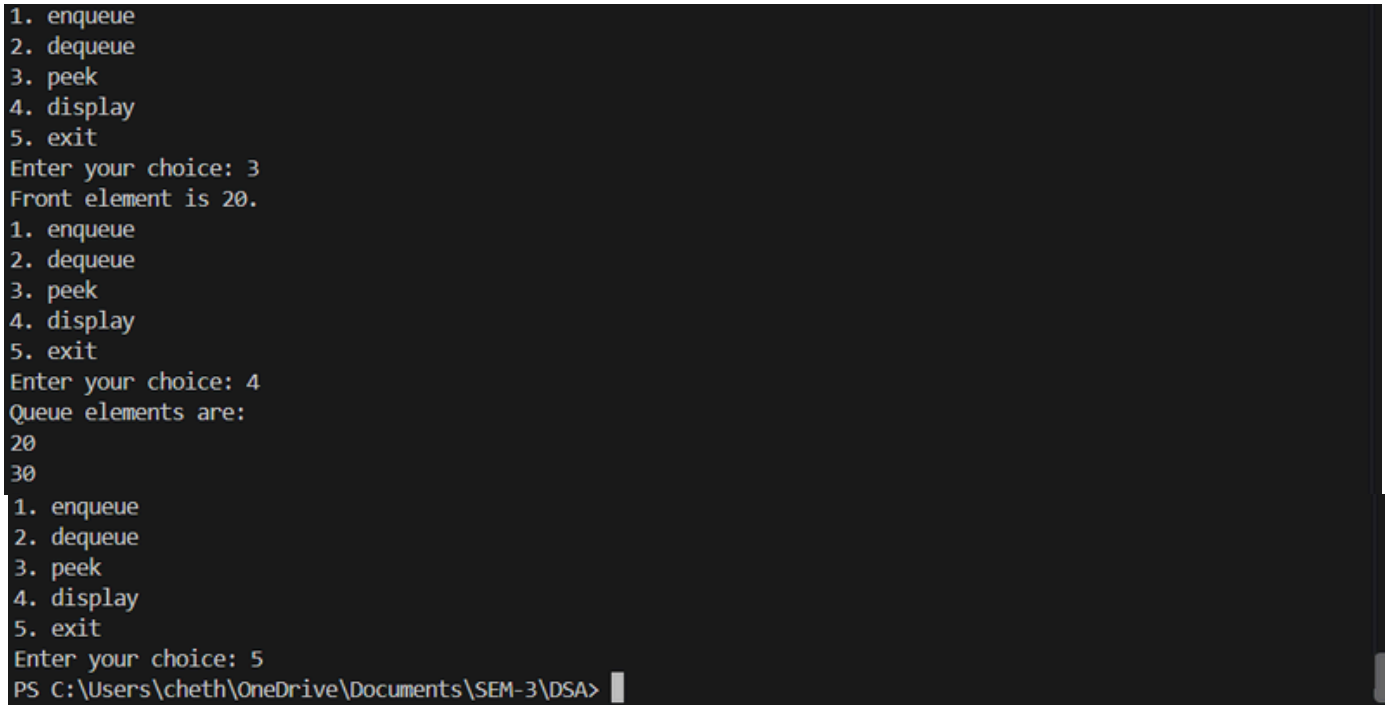
  if choice == 1:
```

```
 data = int(input("Enter value to enqueue:"))
  cq.enqueue(data)
elif choice == 2:
  cq.dequeue()
elif choice == 3:
  cq.peek()
elif choice == 4:
  cq.display()
elif choice == 5:
  break
else:
  print("Invalid choice")
```

Output:

Positive case:





negative case:



Explanation:

- This program is used to implement a Circular Queue using an array.
- The last position of the queue is connected back to the first position, forming a circular structure.
- It supports the enqueue operation to insert an element into the queue.
- It supports the dequeue operation to remove an element from the queue.
- It supports the peek operation to view the front element.
- It displays all elements currently present in the queue.
- It handles Queue Overflow and Queue Underflow conditions.
- It efficiently utilizes memory by reusing empty positions created after dequeue operations.

Week-8

Aim: (a)Write a python program to implement a binary tree with its operations using array.

Code:

```
class BinaryTreeArray:
  def __init__(self,size):
    self.tree = [None]*size
    self.size = size
  #Insert root
  def set_root(self,data):
    self.tree[0] = data
  #Left child
  def set_left(self,parent_index,data):
    child_index = 2*parent_index+1
    if child_index<self.size:
      self.tree[child_index] = data
    else:
      print("Index out of range.")
  #Right child
  def set_right(self,parent_index,data):
    child_index = 2*parent_index+2
    if child_index < self.size:
      self.tree[child_index] = data
    else:
      print("Index out of range.")
  #Preorder Traversal
  def preorder(self,index=0):
    if index >= self.size or self.tree[index] is None:
      return
    print(self.tree[index],end="")
    self.preorder(2*index+1)
    self.preorder(2*index+2)
  #Inorder Traversal
  def inorder(self,index=0):
    if index >= self.size or self.tree[index] is None:
      return
```

```
    self.inorder(2*index+1)
    print(self.tree[index],end=" ")
    self.inorder(2*index+2)
  #Postorder Traversal
  def postorder(self,index=0):
    if index >= self.size or self.tree[index] is None:
      return
    self.postorder(2*index+1)
    self.postorder(2*index+2)
    print(self.tree[index],end=" ")
  #Level order Traversal
  def level_order(self):
    for value in self.tree:
      if value is not None:
        print(value,end=" ")
  #Display
  def display(self):
    print("\nArray Representation")
    for i in range(self.size):
      print(f"Index {i} : {self.tree[i]}")

tree = BinaryTreeArray(7)
#Create tree
tree.set_root('A')
# Children of A
tree.set_left(0, 'B')
tree.set_right(0, 'C')
# Children of B
tree.set_left(1, 'D')
tree.set_right(1, 'E')
# Right child of C
tree.set_right(2, 'F')
tree.display()
print("\nPreorder: ", end="")
tree.preorder()
print("\nInorder: ", end="")
tree.inorder()
```

```
print("\nPostorder: ", end="")
tree.postorder()
print("\nLevel-order: ", end="")
tree.level_order()
```

Output:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-8/(a).py"

Array Representation
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : D
Index 4 : E
Index 5 : None
Index 6 : F

Preorder: ABDECF
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

Explanation:

- Class Creation: BinaryTreeArray defines a binary tree using an array.
- Constructor (__init__): Initializes an array of the specified size with None values and stores the array size.
- Set Root (set_root): Inserts the root node at index 0 of the array.
- Set Left Child (set_left): Inserts the left child at index 2 * parent_index + 1. If the index exceeds the array size, it displays an error message.
- Set Right Child (set_right): Inserts the right child at index 2 * parent_index + 2. If the index exceeds the array size, it displays an error message.
- Preorder Traversal: Visits nodes in the order Root → Left → Right.
- Inorder Traversal: Visits nodes in the order Left → Root → Right.
- Postorder Traversal: Visits nodes in the order Left → Right → Root.
- Level-Order Traversal: Visits the nodes level by level by traversing the array from beginning to end, skipping None values.
- Display (display): Displays each array index and its corresponding value.
- Tree Creation: Creates a binary tree with seven array positions and inserts nodes A, B, C, D, E, and F.
- Function Calls: Calls the display and traversal methods to show the array representation and the four traversal results.

```
print("\nPostorder: ", end="")
tree.postorder()
print("\nLevel-order: ", end="")
tree.level_order()
```

Output:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/1-8/(a).py"

Array Representation
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : D
Index 4 : E
Index 5 : None
Index 6 : F

Preorder: ABDECF
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

Aim: (b)Write a python program to implement a binary tree with its operations using linked list.

Code:

```
class Node:
  def __init__(self,data):
    self.left = None
    self.right = None
    self.data = data

class BinaryTree:
  def __init__(self):
    self.root = None
  def create_tree(self):
    self.root = Node('A')
    # Left and right children of A
    self.root.left = Node('B')
    self.root.right = Node('C')
    # Children of B
    self.root.left.left = Node('D')
    self.root.left.right = Node('E')
    # Right child of C
    self.root.right.right = Node('F')
  # Preorder Traversal
  def preorder(self, node):
    if node is None:
      return
    print(node.data, end=" ")
    self.preorder(node.left)
    self.preorder(node.right)
  # Inorder Traversal
  def inorder(self, node):
    if node is None:
      return
    self.inorder(node.left)
    print(node.data, end=" ")
    self.inorder(node.right)
```

```
  # Postorder Traversal
  def postorder(self, node):
    if node is None:
      return
    self.postorder(node.left)
    self.postorder(node.right)
    print(node.data, end=" ")
  # Level-order Traversal
  def level_order(self):
    if self.root is None:
      return
    queue = [self.root]
    while queue:
      node = queue.pop(0) # Remove first node
      print(node.data, end=" ")
      # Add left child
      if node.left is not None:
        queue.append(node.left)
      # Add right child
      if node.right is not None:
        queue.append(node.right)

tree = BinaryTree()
tree.create_tree()

print("Preorder: ", end="")
tree.preorder(tree.root)

print("\nInorder: ", end="")
tree.inorder(tree.root)

print("\nPostorder: ", end="")
tree.postorder(tree.root)

print("\nLevel-order: ", end="")
tree.level_order()
```

Output:



Explanation:

- Node Class: Defines a node containing three attributes: data, left, and right. The data stores the node value, while left and right store references to its child nodes.
- Constructor (__init__): Initializes the left and right children of each node to None and assigns the given value to data.
- BinaryTree Class: Defines the binary tree and initializes its root to None.
- Create Tree (create_tree): Creates a binary tree with six nodes (A, B, C, D, E, and F) and connects them using left and right references.
- Preorder Traversal: Visits nodes in the order Root → Left → Right.
- Inorder Traversal: Visits nodes in the order Left → Root → Right.
- Postorder Traversal: Visits nodes in the order Left → Right → Root.
- Level-Order Traversal: Visits nodes level by level using a queue. It removes the first node from the queue and adds its existing left and right children.
- Tree Creation: Creates an object of the BinaryTree class and calls create_tree() to construct the tree.
- Function Calls: Calls all four traversal methods to display the nodes in different orders.

Week-9

Aim: (a)Write a python program to implement a binary search tree with its operations and traversals.

Code:

```
class Node:
  def __init__(self,data):
    self.data=data
    self.left=None
    self.right=None
# Insert a node
def insert(root,x):
  if root is None:
    return Node(x)
  if x<root.data:
    root.left=insert(root.left,x)
  elif x>root.data:
    root.right=insert(root.right,x)
  else:
    print("Duplicate value is not allowed")
  return root
# Preorder Traversal
def Preorder(root):
  if root is None:
    return
  print(root.data,end=" ")
  Preorder(root.left)
  Preorder(root.right)
# Inorder Traversal
def Inorder(root):
  if root is None:
    return
  Inorder(root.left)
  print(root.data,end=" ")
  Inorder(root.right)
# Postorder Traversal
def Postorder(root):
```

```
  if root is None:
    return
  Postorder(root.left)
  Postorder(root.right)
  print(root.data,end=" ")
# Find Minimum
def find_min(root):
  if root is None:
    return None
  while root.left is not None:
    root=root.left
  return root
# Find Maximum
def find_max(root):
  if root is None:
    return None
  while root.right is not None:
    root=root.right
  return root
# Search a node
def search(root,data):
  if root is None:
    return None
  if data<root.data:
    return search(root.left,data)
  elif data>root.data:
    return search(root.right,data)
  else:
    return root
# Count total nodes
def count_nodes(root):
  if root is None:
    return 0
  return count_nodes(root.left)+count_nodes(root.right)+1
# Count leaf nodes
def count_leaf(root):
  if root is None:
```

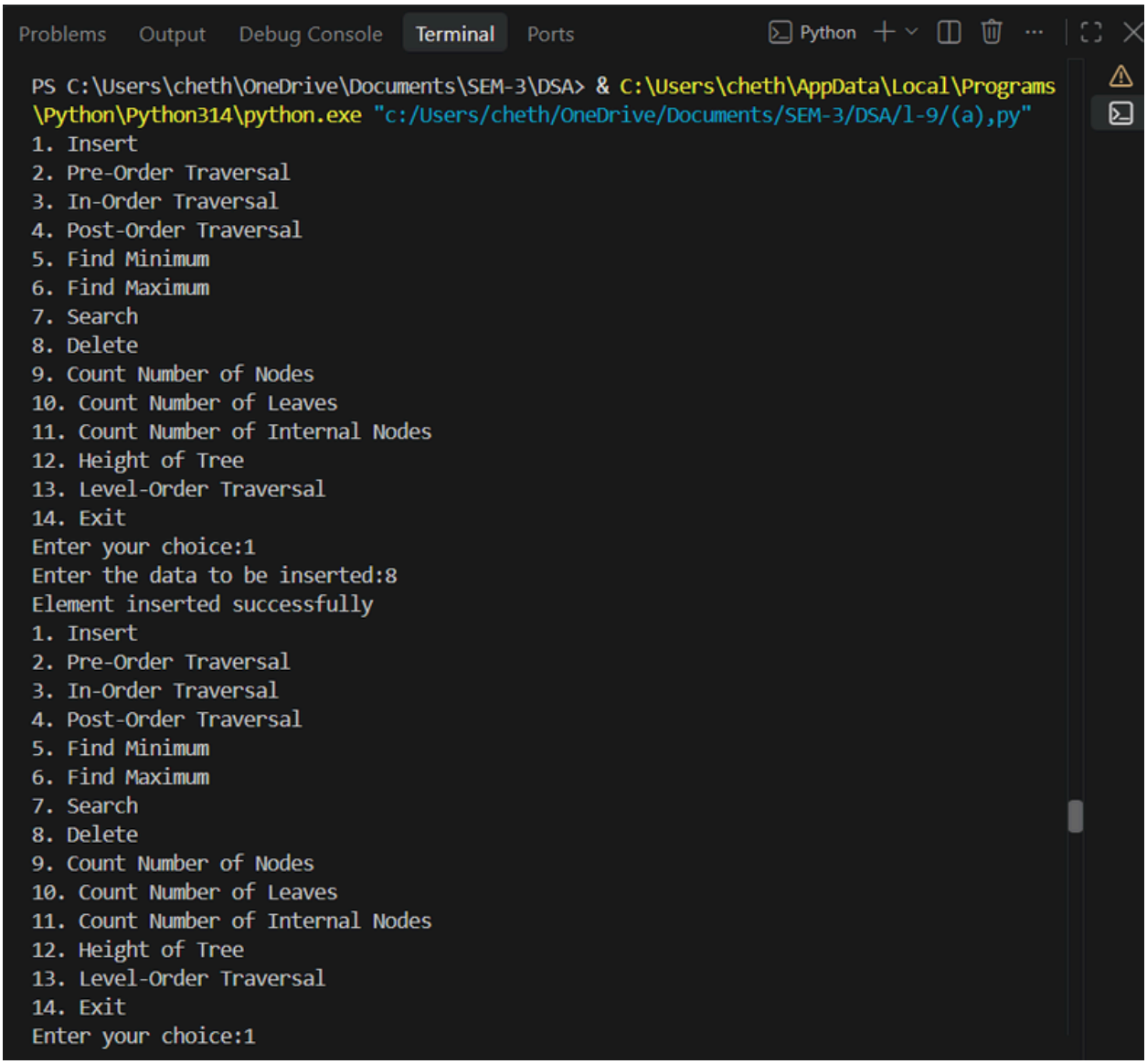
```
    return 0
  if root.left is None and root.right is None:
    return 1
  return count_leaf(root.left)+count_leaf(root.right)
# Count internal nodes
def count_internal(root):
  if root is None:
    return 0
  if root.left is None and root.right is None:
    return 0
  return count_internal(root.left)+count_internal(root.right)+1
# Height of tree
def height(root):
  if root is None:
    return -1
  return max(height(root.left),height(root.right))+1
# Delete a node
def delete(root,x):
  if root is None:
    return None
  if x<root.data:
    root.left=delete(root.left,x)
  elif x>root.data:
    root.right=delete(root.right,x)
  else:
    # No children
    if root.left is None and root.right is None:
      return None
    # Only right child
    elif root.left is None:
      return root.right
    # Only left child
    elif root.right is None:
      return root.left
    # Two children
    successor=find_min(root.right)
    root.data=successor.data
```

```
    root.right=delete(root.right,successor.data)
  return root
# Level-order Traversal
def level_order(root):
  if root is None:
    return
  queue=[root]
  while queue:
    node=queue.pop(0)
    print(node.data,end=" ")
    if node.left is not None:
      queue.append(node.left)
    if node.right is not None:
      queue.append(node.right)
# Display tree level by level
def display(root,level=0):
  if root is None:
    return
  queue=[(root,0)]
  while queue:
    node,current_level=queue.pop(0)
    if current_level==level:
      print(node.data,end=" ")
    if node.left is not None:
      queue.append((node.left,current_level+1))
    if node.right is not None:
      queue.append((node.right,current_level+1))
# Menu-driven Program
root = None
while True:
  print("1. Insert")
  print("2. Pre-Order Traversal")
  print("3. In-Order Traversal")
  print("4. Post-Order Traversal")
  print("5. Find Minimum")
  print("6. Find Maximum")
  print("7. Search")
```

```
print("8. Delete")
print("9. Count Number of Nodes")
print("10. Count Number of Leaves")
print("11. Count Number of Internal Nodes")
print("12. Height of Tree")
print("13. Level-Order Traversal")
print("14. Exit")
choice=int(input("Enter your choice:"))
if choice==1:
  data=int(input("Enter the data to be inserted:"))
  root=insert(root,data)
  print("Element inserted successfully")
elif choice==2:
  print("Preorder Traversal:",end="")
  if root is None:
    print("Tree is Empty")
  else:
    Preorder(root)
    print()
elif choice==3:
  print("Inorder Traversal:",end="")
  if root is None:
    print("Tree is Empty")
  else:
    Inorder(root)
    print()
elif choice==4:
  print("Postorder Traversal:",end="")
  if root is None:
    print("Tree is Empty")
  else:
    Postorder(root)
    print()
elif choice==5:
  node=find_min(root)
  if node is None:
    print("Tree is Empty")
```

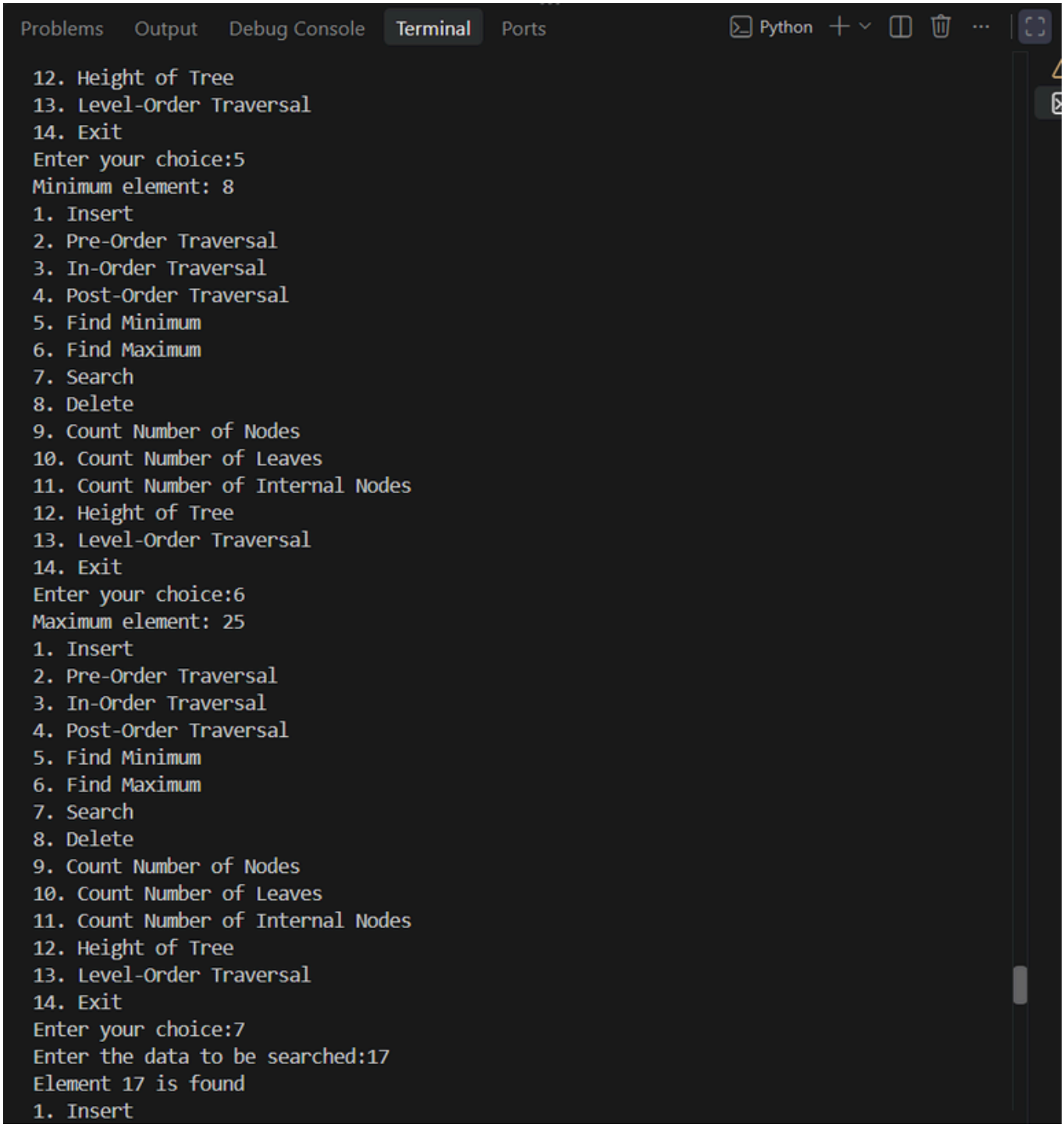
```
  else:
    print("Minimum element:",node.data)
elif choice==6:
  node=find_max(root)
  if node is None:
    print("Tree is Empty")
  else:
    print("Maximum element:",node.data)
elif choice==7:
  data=int(input("Enter the data to be searched:"))
  node=search(root,data)
  if node is None:
    print("Element",data,"is not found")
  else:
    print("Element",data,"is found")
elif choice==8:
  data=int(input("Enter the data to be deleted:"))
  root=delete(root,data)
elif choice==9:
  print("Number of nodes:",count_nodes(root))
elif choice==10:
  print("Number of leaf nodes:",count_leaf(root))
elif choice==11:
  print("Number of internal nodes:",count_internal(root))
elif choice==12:
  print("Height of BST:",height(root))
elif choice==13:
  print("Level order Traversal:",end="")
  if root is None:
    print("Tree is Empty")
  else:
    level_order(root)
    print()
elif choice==14:
  print("Program terminated.")
  break
else:
  print("Enter a valid choice.")
```

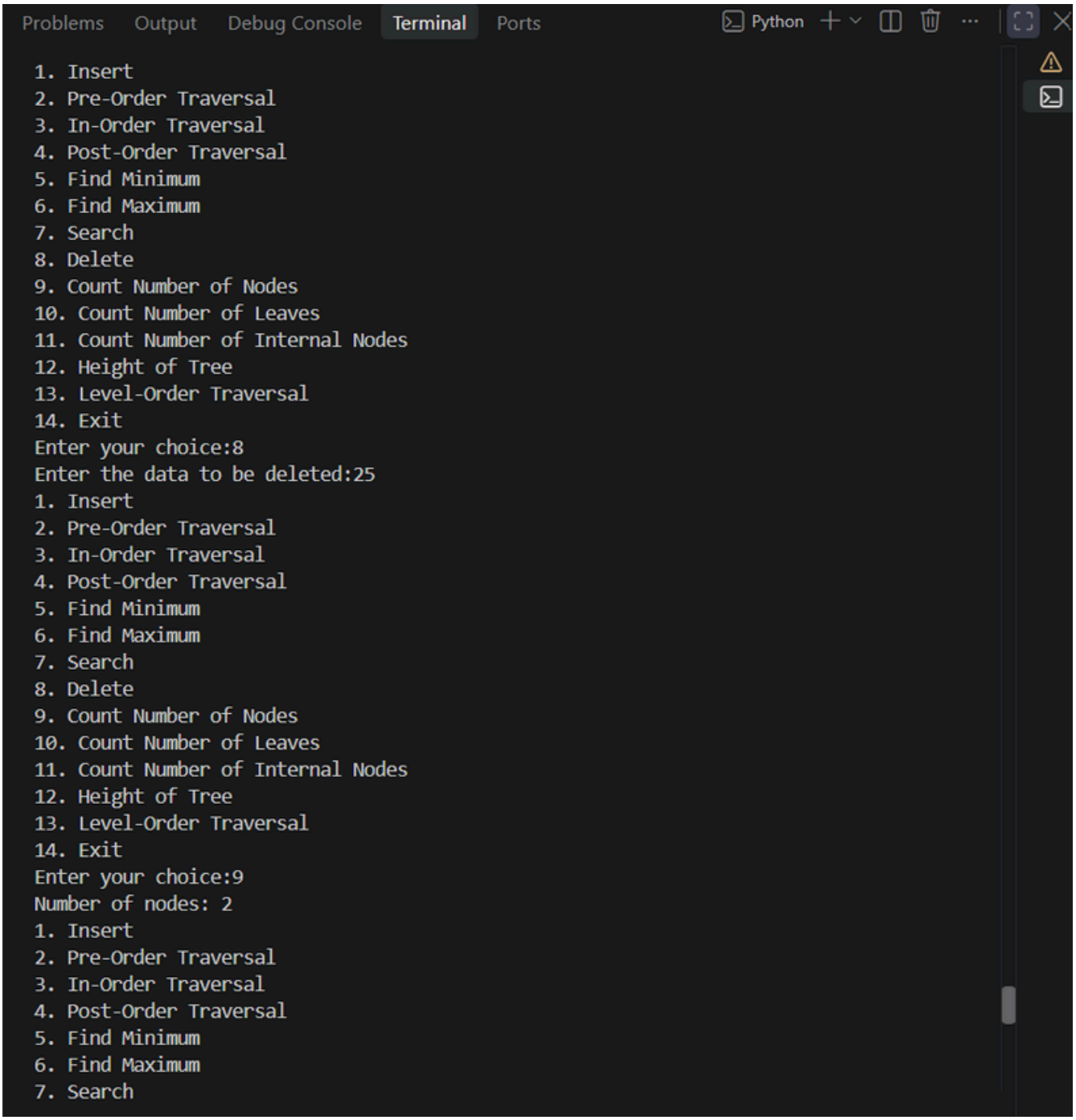
Output:
positive case:

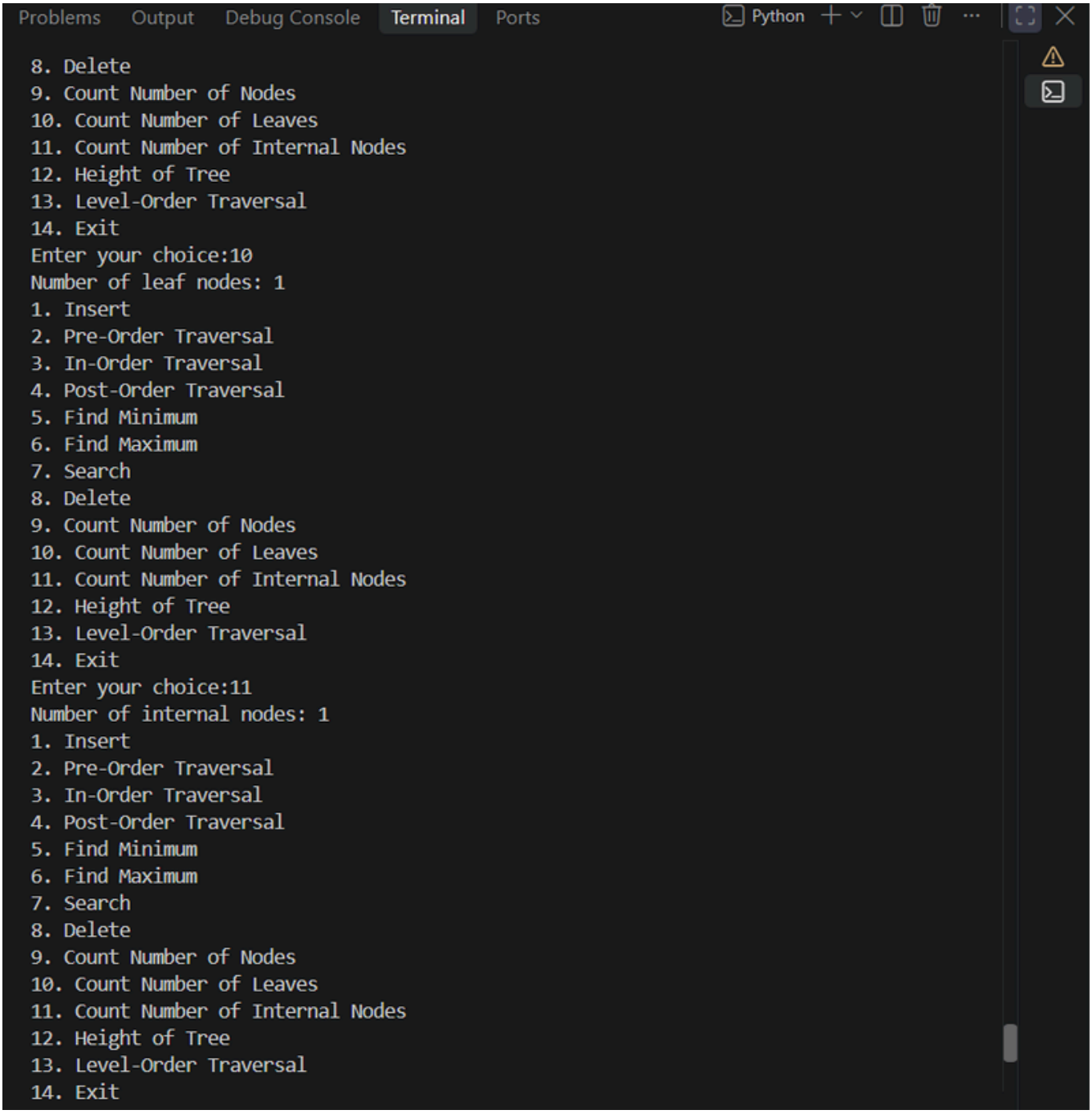


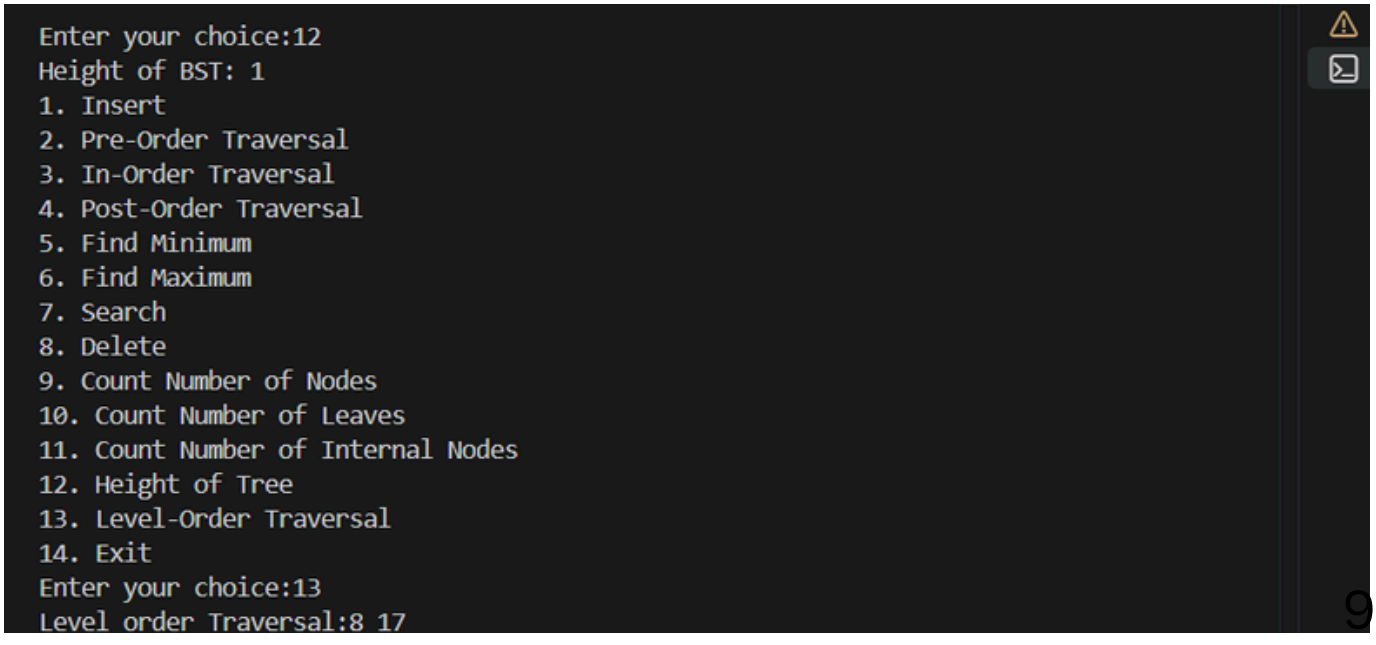

Problems
Output
Debug Console
Terminal
Ports
Python
Enter the data to be inserted:17
Element inserted successfully
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:1
Enter the data to be inserted:25
Element inserted successfully
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:2
Preorder Traversal:8 17 25
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal

```
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:3
Inorder Traversal:8 17 25
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:4
Postorder Traversal:25 17 8
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
```


Problems
Output
Debug Console
Terminal
Ports
Python
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:5
Minimum element: 8
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:6
Maximum element: 25
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:7
Enter the data to be searched:17
Element 17 is found
1. Insert


Problems
Output
Debug Console
Terminal
Ports
Python
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:8
Enter the data to be deleted:25
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:9
Number of nodes: 2
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search


Problems
Output
Debug Console
Terminal
Ports
Python
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:10
Number of leaf nodes: 1
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:11
Number of internal nodes: 1
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit


Enter your choice:12
Height of BST: 1
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:13
Level order Traversal:8 17

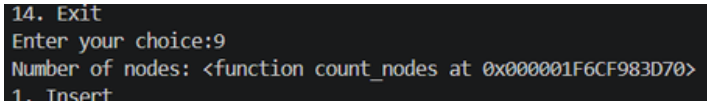
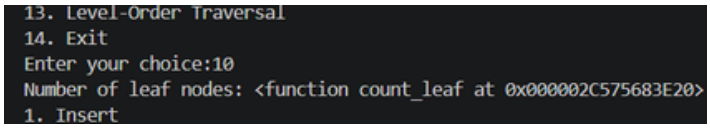
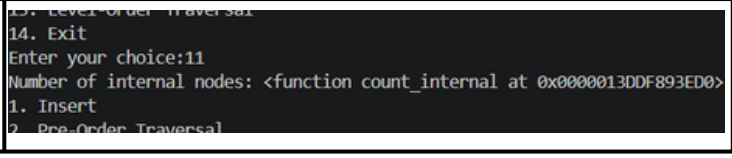
negative case:

```
1. Insert
2. Pre-Order Traversal
3. In-Order Traversal
4. Post-Order Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Number of Nodes
10. Count Number of Leaves
11. Count Number of Internal Nodes
12. Height of Tree
13. Level-Order Traversal
14. Exit
Enter your choice:14
Program terminated.                                & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-8/(b).py"
```

Explanation:

- Node Class: Creates a node with data, left child, and right child.
- Insert: Inserts elements according to BST rules; smaller values go left and larger values go right.
- Preorder: Traverses in the order Root → Left → Right.
- Inorder: Traverses in the order Left → Root → Right.
- Postorder: Traverses in the order Left → Right → Root.
- Find Minimum: Finds the leftmost node.
- Find Maximum: Finds the rightmost node.
- Search: Searches for an element in the BST.
- Count Nodes: Returns the total number of nodes.
- Count Leaves: Counts nodes without children.
- Count Internal Nodes: Counts nodes having at least one child.
- Height: Calculates the longest path from the root to a leaf.
- Delete: Deletes a node based on whether it has zero, one, or two children.
- Level-Order Traversal: Visits nodes level by level using a queue.
- Display: Prints nodes at a specified level.
- Menu-Driven Program: Allows the user to select and perform BST operations.
- Exit: Terminates the program when option 14 is selected

Error Table:

s.no	Error	Correction
1.		count_nodes(root)
2.		count_leaf(root)
3.		count_internal(root)

Week-10

Aim: (a)Write a python program to implement AVL tree with all its operations.

Code:

```
class Node:
  def __init__(self, data):
    self.data = data
    self.left = None
    self.right = None
    self.height = 1
class AVLTree:
  def height(self, root):
    if root is None:
      return 0
    return root.height
  def balance(self, root):
    if root is None:
      return 0
    return self.height(root.left) - self.height(root.right)
  def right_rotate(self, y):
    x = y.left
    t = x.right
    x.right = y
    y.left = t
    y.height = 1 + max(self.height(y.left), self.height(y.right))
    x.height = 1 + max(self.height(x.left), self.height(x.right))
    return x
  def left_rotate(self, x):
    y = x.right
    t = y.left
    y.left = x
    x.right = t
    x.height = 1 + max(self.height(x.left), self.height(x.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))
    return y
  def insert(self, root, data):
```

```
    if root is None:
      return Node(data)
    if data < root.data:
      root.left = self.insert(root.left, data)
    elif data > root.data:
      root.right = self.insert(root.right, data)
    else:
      return root
    root.height = 1 + max(self.height(root.left), self.height(root.right))
    b = self.balance(root)
    if b > 1 and data < root.left.data:
      return self.right_rotate(root)
    if b < -1 and data > root.right.data:
      return self.left_rotate(root)
    if b > 1 and data > root.left.data:
      root.left = self.left_rotate(root.left)
      return self.right_rotate(root)
    if b < -1 and data < root.right.data:
      root.right = self.right_rotate(root.right)
      return self.left_rotate(root)
    return root
  def search(self, root, data):
    if root is None:
      return False
    if root.data == data:
      return True
    elif data < root.data:
      return self.search(root.left, data)
    else:
      return self.search(root.right, data)
  def find_min(self, root):
    current = root
    while current.left is not None:
      current = current.left
    return current
  def delete(self, root, data):
    if root is None:
```

```
        return root
    if data < root.data:
        root.left = self.delete(root.left, data)
    elif data > root.data:
        root.right = self.delete(root.right, data)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            temp = self.find_min(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)
    root.height = 1 + max(self.height(root.left), self.height(root.right))
    b = self.balance(root)
    if b > 1 and self.balance(root.left) >= 0:
        return self.right_rotate(root)
    if b > 1 and self.balance(root.left) < 0:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)
    if b < -1 and self.balance(root.right) <= 0:
        return self.left_rotate(root)
    if b < -1 and self.balance(root.right) > 0:
        root.right = self.right_rotate(root.right)
        return self.left_rotate(root)
    return root
def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)
def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)
```

```
  def postorder(self, root):
    if root:
      self.postorder(root.left)
      self.postorder(root.right)
      print(root.data, end=" ")
  def level_order(self, root):
    if root is None:
      return
    queue = [root]
    while queue:
      node = queue.pop(0)
      print(node.data, end=" ")
      if node.left:
        queue.append(node.left)
      if node.right:
        queue.append(node.right)
avl = AVLTree()
root = None

while True:
  print("Name:T.chethana Nagasai")
  print("Roll No: AV.SC.U4CSE25104")
  print("1. Insert")
  print("2. Search")
  print("3. Delete")
  print("4. Inorder Traversal")
  print("5. Preorder Traversal")
  print("6. Postorder Traversal")
  print("7. Level Order Traversal")
  print("8. Exit")

  choice = int(input("Enter your choice: "))

  if choice == 1:
    data = int(input("Enter value to insert: "))
    root = avl.insert(root, data)
    print("Node inserted successfully.")
```

```
elif choice == 2:
  data = int(input("Enter value to search: "))
  if avl.search(root, data):
    print("Element found.")
  else:
    print("Element not found.")
elif choice == 3:
  data = int(input("Enter value to delete: "))
  root = avl.delete(root, data)
  print("Node deleted successfully.")
elif choice == 4:
  print("Inorder Traversal:")
  avl.inorder(root)
elif choice == 5:
  print("Preorder Traversal:")
  avl.preorder(root)
elif choice == 6:
  print("Postorder Traversal:")
  avl.postorder(root)
elif choice == 7:
  print("Level Order Traversal:")
  avl.level_order(root)
elif choice == 8:
  print("Program terminated.")
  break
else:
  print("Invalid choice.")
```

Output:

positive case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-10/(a).py"
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter value to insert: 10
Node inserted successfully.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter value to insert: 20
Node inserted successfully.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
```

```
Enter value to insert: 30
Node inserted successfully.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter value to insert: 40
Node inserted successfully.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 2
Enter value to search: 3
Element not found.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 3
```

```
Enter value to delete: 40
Node deleted successfully.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 4
Inorder Traversal:
10 20 30 Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 5
Preorder Traversal:
20 10 30 Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 6
Postorder Traversal:
10 30 20 Name:T.chethana Nagasai
```

```
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 7
Level Order Traversal:
20 10 30 Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 8
Program terminated.
```

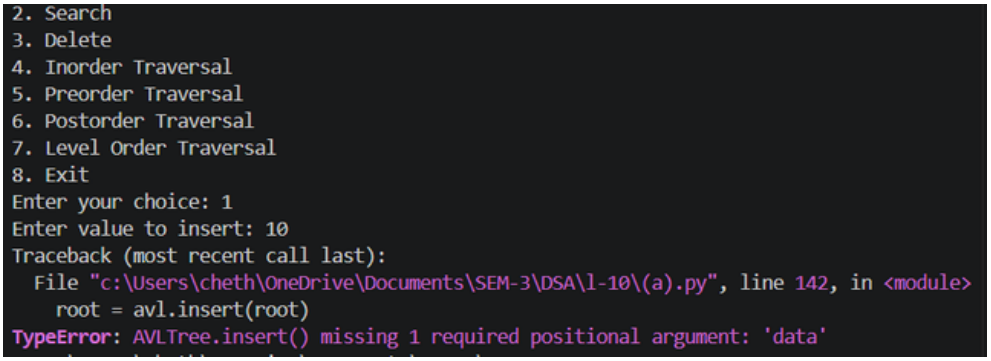
negative case:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-10/(a).py"
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 9
Invalid choice.
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice:
```

Explanation:

- AVL Tree: Creates a self-balancing binary search tree in which the balance factor of each node is maintained between -1 and +1.
- Insertion: Inserts new nodes according to binary search tree rules and performs rotations whenever the tree becomes unbalanced.
- Searching and Deletion: Searches for a given element and deletes the required node while maintaining the AVL tree balance.
- Rotations: Uses left rotation, right rotation, left-right rotation and right-left rotation to balance the tree.
- Tree Traversals: Displays elements using inorder, preorder, postorder and level order traversal methods.

Error Table:

s.no	Error	Correction
1.		insert(root,data)

Week-11

Aim: (a)Write a python program to implement hashing(separate chaining, Linear probing and Quadratic probing).

Code:

```
size = 10
chain = [[] for i in range(size)]
def separate_chaining(key):
  index = key % size
  chain[index].append(key)

linear = [None] * size

def linear_probing(key):
  index = key % size
  while linear[index] is not None:
    index = (index + 1) % size
  linear[index] = key

quadratic = [None] * size

def quadratic_probing(key):
  index = key % size
  i = 0
  while i < size:
    new_index = (index + i * i) % size
    if quadratic[new_index] is None:
      quadratic[new_index] = key
      return
    i += 1
  print("Hash table is full")

n = int(input("Enter number of elements: "))
keys = []

for i in range(n):
  key = int(input("Enter key: "))
```

```
    keys.append(key)
    separate_chaining(key)
    linear_probing(key)
    quadratic_probing(key)

print("Name:T.chethana Nagasai")
print("Roll No: AV.SC.U4CSE25104")
print("\nSeparate Chaining:")
for i in range(size):
  print(i, ":", chain[i])
print("\nLinear Probing:")
print(linear)
print("\nQuadratic Probing:")
print(quadratic)
```

output:

```
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA> & C:\Users\cheth\AppData\Local\Programs
\Python\Python314\python.exe "c:/Users/cheth/OneDrive/Documents/SEM-3/DSA/l-11/(a).py"
Enter number of elements: 3
Enter key: 10
Enter key: 20
Enter key: 30
Name:T.chethana Nagasai
Roll No: AV.SC.U4CSE25104

Separate Chaining:
0 : [10, 20, 30]
1 : []
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []

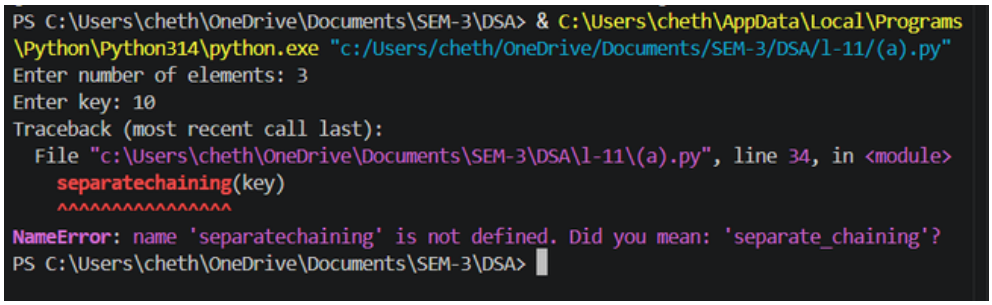
Linear Probing:
[10, 20, 30, None, None, None, None, None, None, None]

Quadratic Probing:
[10, 20, None, None, 30, None, None, None, None, None]
PS C:\Users\cheth\OneDrive\Documents\SEM-3\DSA>
```

Explanation:

- Hash Table: A hash table of size 10 is created to store the keys using the hash function key % size.
- Separate Chaining: If two keys get the same index, they are stored in the same list at that index.
- Linear Probing: If the calculated index is already occupied, the next available position is checked sequentially.
- Quadratic Probing: If a collision occurs, the next position is calculated using the formula (index + i * i) % size.
- Display: Finally, all three hash tables are displayed to compare how separate chaining, linear probing and quadratic probing handle collisions.

Error Table:

s.no	Error	Correction
1.		separate_chaining